



Agentic AI Engineering

Zero to Hero — Complete Lecture Notes

OpenAI Agents SDK

CrewAI

LangGraph

AutoGen

MCP

8 Real-World Projects

6 Weeks Curriculum

Beginner Friendly

Based on the course by **Ed Donner** — AI Startup Co-Founder & Gen AI Instructor

Self-contained notes — no prior AI agent experience required



Table of Contents

Your roadmap through this guide

- 01 What is Agentic AI? (The Big Picture)
- 02 How AI Agents Think — Core Concepts
- 03 Week 1 — Foundations & Setup
- 04 Week 2 — OpenAI Agents SDK
- 05 Week 3 — CrewAI
- 06 Week 4 — LangGraph
- 07 Week 5 — AutoGen (AG2)
- 08 Week 6 — MCP & Capstone
- 09 All 8 Projects at a Glance
- 10 Framework Comparison & When to Use What
- 11 Glossary of Key Terms

01 — What is Agentic AI? (The Big Picture)

Start here — understand the "why" before the "how"

The Evolution: Chatbot → LLM → Agent

Before we dive into code and frameworks, let's understand what makes an AI "agentic" and why it matters.

Type	What it does	Example	Limitation
Chatbot	Responds to messages with pre-written rules	FAQ bot on a website	Can't think, only matches patterns
LLM App	Uses a language model to generate smart responses	ChatGPT conversation	One question → one answer, no memory of tasks
AI Agent	Plans, takes actions, uses tools, loops until done	Research agent that browses web + writes report	More complex to build, but far more powerful

REAL-WORLD ANALOGY

Chatbot = A vending machine. You press a button, you get a fixed result.

LLM App = A knowledgeable friend who answers your questions.

AI Agent = A personal assistant who takes your goal ("plan my trip to Paris"), breaks it into steps, books flights, checks weather, and reports back — all on their own.

What Makes Something an "Agent"?

According to Anthropic's research, an AI system is "agentic" when the LLM **controls the workflow** — it decides what to do next, not just what to say.

The 4 Pillars of an AI Agent

-  **Reasoning** — The agent thinks through a problem step by step
-  **Tool Use** — It can call external tools (search web, run code, send email)
-  **Looping** — It checks its own output and tries again if needed
-  **Collaboration** — Multiple agents can work together on complex tasks

Traditional LLM vs Agentic AI — Side by Side

Aspect	Traditional LLM App	Agentic AI
Flow control	You (the developer) decide every step	The LLM decides what to do next
Tools	Fixed, pre-defined	Dynamic — agent picks which tool to use
Memory	Usually none (stateless)	Can remember context across steps
Error handling	Crashes or returns wrong answer	Can retry, self-correct, ask for help
Complexity	Simple Q&A or generation	Multi-step, multi-tool, multi-agent tasks

 **Key Insight:** The shift from LLM apps to Agentic AI is like the shift from calculators to computers. A calculator does one thing. A computer can run any program. Agents can tackle any task — if designed well.

Why Does This Matter Right Now?

In 2025-2026, AI agents went from research curiosity to production reality. Companies are using them to:

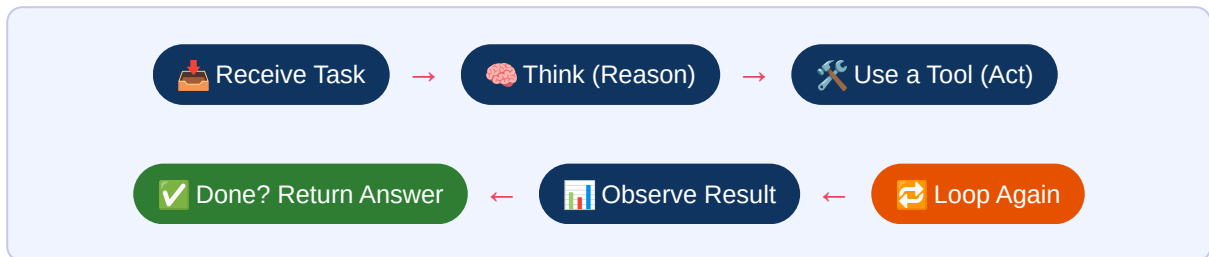
- Automate sales outreach (SDR agents writing personalized emails)
- Conduct deep research (replacing hours of manual work)
- Build software (engineering teams of AI agents)
- Trade financial markets (autonomous trading floors)
- Browse the web on your behalf (like OpenAI's Operator)

02 — How AI Agents Think — Core Concepts

The mental model you need before touching any framework

The ReAct Loop — How Every Agent Works

Almost every AI agent follows a pattern called **ReAct** (Reason + Act). Here's the loop:



DETECTIVE ANALOGY

Think of an agent like Sherlock Holmes. He gets a case (task), thinks about what clues to look for (reason), goes and finds them (act with tools), observes what he found, and keeps investigating until he solves the mystery. He doesn't just guess once — he loops until he has the answer.

What Are "Tools"?

Tools are functions the agent can call to interact with the outside world. Without tools, an agent is just a chatbot. With tools, it becomes powerful.

Tool Type	What it does	Example
Web Search	Searches the internet	Find latest stock prices
Code Execution	Runs Python/JS code	Calculate statistics, process data
File Read/Write	Reads or saves files	Read a CSV, save a report
API Call	Calls external services	Send email, post to Slack
Browser Control	Controls a web browser	Fill forms, click buttons
Database Query	Reads/writes databases	Look up customer records

The 5 LLM Workflow Design Patterns (Anthropic)

Before building agents, understand these 5 patterns. Every agent framework uses some combination of them:

1. Prompt Chaining

Break a big task into sequential steps. Output of step 1 becomes input of step 2.

Example: Research topic → Summarize findings → Write article → Proofread

2. Routing

Classify the input first, then send it to the right specialist agent.

Example: Customer complaint → Route to billing agent OR technical support agent

3. Parallelization

Run multiple agents simultaneously on different parts of a task, then combine results.

Example: Research 5 companies at the same time, then compare

4. Orchestrator-Worker

One "boss" agent breaks down the task and assigns sub-tasks to "worker" agents.

Example: Manager agent assigns coding, testing, and documentation to separate agents

5. Evaluator-Optimizer

One agent generates output, another evaluates it, and the first improves based on feedback. Loops until quality is good enough.

Example: Writer agent drafts → Critic agent scores → Writer improves → Repeat

Memory Types in AI Agents

Memory Type	What it is	Analogy
In-Context	What's in the current conversation window	Your working memory right now
External (Short-term)	Stored in a database, retrieved when needed	Notes you wrote today
External (Long-term)	Persistent storage across sessions	Your diary from years ago
Semantic (Vector)	Stored by meaning, retrieved by similarity	Searching your notes by topic

Single Agent vs Multi-Agent Systems

When to use a Single Agent

- Task is focused and well-defined
- One set of tools is enough
- Simpler to debug and maintain

When to use Multiple Agents

- Task is too complex for one agent's context window
- Different parts need different expertise/tools
- Parallel processing speeds things up
- You want checks and balances (one agent reviews another)

03 — Week 1: Foundations & Setup

Building your mental model and development environment

What You'll Learn This Week

The "why" of agentic AI, environment setup, and your first agentic workflow

Development Environment Setup

You need 3 things to get started:

Tool	What it is	Why use it
Cursor IDE	AI-powered code editor (built on VS Code)	AI helps you write code faster — perfect for learning
UV Package Manager	Fast Python environment manager (written in Rust)	10x faster than pip/conda for installing packages
Python 3.12+	Programming language	All AI frameworks are Python-first

API Keys — What You Need

 **Important:** Never hardcode API keys in your code! Always use environment variables (.env files). This prevents accidentally sharing your keys on GitHub.

```
# .env file (never commit this to GitHub!)
OPENAI_API_KEY=sk-...your-key-here...
ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=...

# In your Python code, load it like this:
from dotenv import load_dotenv
import os

load_dotenv() # reads the .env file
api_key = os.getenv("OPENAI_API_KEY")
```

Python Concepts You Need to Know

The course uses some Python features that beginners might not know. Here's a quick primer:

async / await — Asynchronous Programming

Normal Python code runs one line at a time. Async code can do multiple things "at once" — perfect for agents that need to call multiple APIs simultaneously.

```
# NORMAL (synchronous) – waits for each step
result1 = search_web("topic A")    # waits 2 seconds
result2 = search_web("topic B")    # waits 2 more seconds = 4 seconds total

# ASYNC – runs both at the same time!
import asyncio

async def run_research():
    result1, result2 = await asyncio.gather(
        search_web("topic A"),
        search_web("topic B")
    )
    # Both run simultaneously = ~2 seconds total!
    return result1, result2
```



COFFEE SHOP ANALOGY

Synchronous: One barista makes one coffee, then starts the next. Everyone waits in line.

Asynchronous: The barista starts coffee #1, while it brews starts coffee #2, while that brews starts #3. Much faster!



Your First Agentic Workflow

The simplest possible agent — an LLM that decides what to do next:

```
from openai import OpenAI

client = OpenAI()

# Give the LLM a task and let it decide the approach
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content":
            "You are a business analyst. Choose which industry to analyze and explain your reasoning."},
        {"role": "user", "content": "Pick the most promising sector for investment in 2025 and analyze it."}
    ]
)

# The LLM autonomously chose the sector – that's the "agentic" part!
print(response.choices[0].message.content)
```

💡 **What makes this "agentic"?** The LLM made a decision (which sector to pick) that wasn't pre-programmed. The developer didn't specify the sector — the AI chose it. That's the beginning of autonomy.

04 — Week 2: OpenAI Agents SDK

Build production-grade agents with OpenAI's official framework

What is the OpenAI Agents SDK?

Released March 2025 — OpenAI's official, lightweight framework for building agents

OpenAI Agents SDK

"Elegant and flexible — just enough structure without heavyweight abstractions"

Think of it as a thin, clean layer on top of the OpenAI API that adds the key ingredients for agents: tools, handoffs between agents, guardrails, and tracing.

✓ Easy to learn

✓ Great for OpenAI users

✓ Built-in tracing

⚠ OpenAI-first

The 5 Core Building Blocks

Concept	What it is	Simple Analogy
Agent	An AI with a name, instructions, and tools	An employee with a job description
Tool	A function the agent can call	The employee's equipment/software
Handoff	One agent passes control to another	Transferring a call to a specialist
Guardrail	Rules that constrain agent behavior	Company policy / compliance rules
Trace	A log of everything the agent did	An audit trail / activity log

Building Your First Agent — Step by Step

```
from agents import Agent, Runner, function_tool
import asyncio

# Step 1: Define a tool (a function the agent can use)
@function_tool
def search_web(query: str) -> str:
    """Search the web for information about a topic."""
    # In real code, this would call a search API
    return f"Search results for: {query}"

# Step 2: Create an agent with instructions and tools
research_agent = Agent(
    name="Research Assistant",
```

```

        instructions="""You are a thorough research assistant.
        When given a topic, search for information and provide a comprehensive
        summary.
        Always cite your sources."""
        tools=[search_web] # Give the agent access to the tool
    )

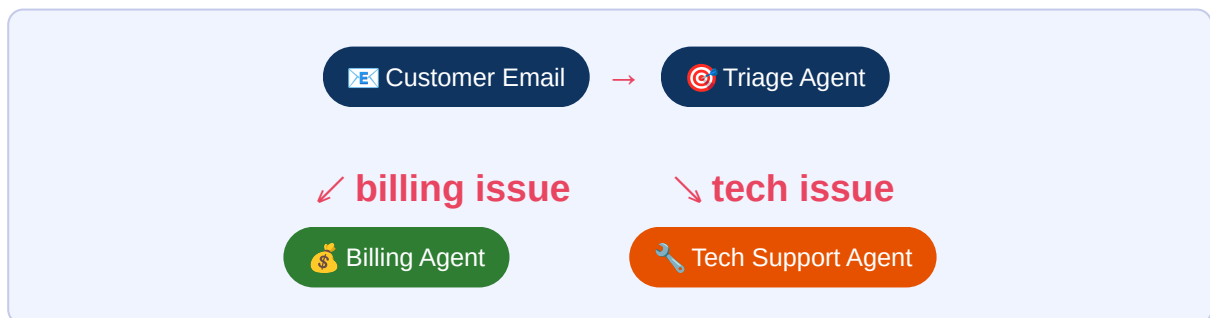
# Step 3: Run the agent
async def main():
    result = await Runner.run(
        research_agent,
        "Research the latest developments in quantum computing"
    )
    print(result.final_output)

asyncio.run(main())

```

🤝 Agent Handoffs — Teamwork Between Agents

Handoffs let you build specialist agents that work together. When one agent can't handle something, it passes to the right specialist.



```

from agents import Agent, handoff

# Specialist agents
billing_agent = Agent(
    name="Billing Specialist",
    instructions="Handle all billing and payment questions."
)

tech_agent = Agent(
    name="Tech Support",
    instructions="Handle all technical issues and bugs."
)

# Triage agent that routes to specialists
triage_agent = Agent(
    name="Customer Service Triage",
    instructions="""Classify customer issues and route to the right specialist.
    - Billing/payment issues → transfer to Billing Specialist
    - Technical problems → transfer to Tech Support""",

```

```
handoffs=[billing_agent, tech_agent] # Can hand off to these agents
)
```

Guardrails — Keeping Agents Safe

What are Guardrails?

Guardrails are validation checks that run before or after agent actions. They prevent agents from doing harmful, off-topic, or incorrect things.

```
from agents import Agent, GuardrailFunctionOutput, input_guardrail

# Define a guardrail that blocks off-topic requests
@input_guardrail
async def topic_guardrail(ctx, agent, input):
    # Check if the input is about finance (our agent's domain)
    is_finance_related = "stock" in input.lower() or "invest" in input.lower()
    return GuardrailFunctionOutput(
        output_info="Topic check",
        tripwire_triggered=not is_finance_related # Block if NOT finance
    )

finance_agent = Agent(
    name="Finance Advisor",
    instructions="Provide financial advice and stock analysis.",
    input_guardrails=[topic_guardrail] # Apply the guardrail
)
```

Tracing — Understanding What Your Agent Did

Tracing automatically records every step your agent takes. You can view traces in the OpenAI dashboard to debug issues.

What Tracing Records

- Every message sent to the LLM
- Every tool call made (and its result)
- Every handoff between agents
- Time taken for each step
- Token usage and costs

Project 1: Career Digital Twin

PROJECT 1

Career Digital Twin — Your AI Representative

What it does: An agent that represents you to potential employers. It knows your skills, experience, and goals, and can answer questions on your behalf.

Key concepts used: Agent with custom instructions, tools to look up your resume data, guardrails to stay professional

Real-world value: Imagine sending this agent to "attend" job fairs or respond to recruiter emails automatically



Project 2: SDR (Sales Development Representative) Agent

PROJECT 2

SDR Agent — Automated Sales Outreach

What it does: Researches potential customers, crafts personalized outreach emails, and sends them automatically.

Key concepts used: Web search tool, email sending tool, multi-step workflow (research → draft → send)

Real-world value: Replaces hours of manual sales research and email writing



Project 3: Deep Research Agent

PROJECT 3

Deep Research — Your Personal Research Team

What it does: A team of agents that conducts extensive research on any topic — like having a team of analysts working for you.

Architecture: Planner agent → Multiple researcher agents (parallel) → Synthesizer agent → Report writer agent

Key concepts used: Multi-agent handoffs, parallelization pattern, evaluator-optimizer loop

05 — Week 3: CrewAI

Build teams of AI agents that work together like a real company

What is CrewAI?

The "low-code fan favorite" — build multi-agent teams with minimal code

CrewAI

"Orchestrate role-playing AI agents like a real team — fast and intuitive"

CrewAI lets you define agents as if they were employees with job titles, goals, and backstories. You then give them tasks and let them collaborate. It's the fastest way to get a multi-agent system running.

✓ Fastest to prototype

✓ Intuitive role-based design

✓ Visual Studio editor

⚠ Less control than LangGraph

The CrewAI Mental Model — Think Like a Company

COMPANY ANALOGY

Imagine you're building a startup. You hire people with specific roles (CEO, researcher, writer, coder). You give them tasks. They work together to deliver results. CrewAI works exactly the same way — but with AI agents instead of humans.

CrewAI Concept	Real-World Equivalent	What you define
Agent	An employee	Role, goal, backstory, tools
Task	A work assignment	Description, expected output, assigned agent
Crew	The team	List of agents + tasks + process type
Process	How work flows	Sequential or Hierarchical
Tool	Software/equipment	Functions agents can call

Building a CrewAI System — Step by Step

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool
```

```

# ---- STEP 1: Define your agents (like hiring employees) ----

researcher = Agent(
    role="Senior Research Analyst",
    goal="Find accurate, up-to-date information on any topic",
    backstory="""You are an expert researcher with 10 years of experience.
    You are known for your thorough analysis and attention to detail.""",
    tools=[SerperDevTool()], # Can search the web
    verbose=True # Show what the agent is thinking
)

writer = Agent(
    role="Content Writer",
    goal="Write clear, engaging content based on research",
    backstory="""You are a skilled writer who transforms complex research
    into readable, compelling articles.""",
    verbose=True
)

# ---- STEP 2: Define tasks (like assigning work) ----

research_task = Task(
    description="Research the top 5 AI trends in 2025. Find specific examples
    and data.",

    expected_output="A detailed report with 5 AI trends, each with examples and
    statistics.",
    agent=researcher # Assign to the researcher
)

writing_task = Task(
    description="Write a blog post based on the research. Make it engaging for
    a general audience.",

    expected_output="A 500-word blog post with an introduction, 5 sections, and
    conclusion.",
    agent=writer,
    context=[research_task] # Writer gets the researcher's output
)

# ---- STEP 3: Assemble the crew and run ----

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process=Process.sequential # Research first, then write
)

result = crew.kickoff()
print(result)

```

Sequential vs Hierarchical Process

Process Type	How it works	When to use
Sequential	Tasks run one after another in order. Output of task 1 feeds into task 2.	Linear workflows: research → write → review
Hierarchical	A manager agent oversees worker agents, assigns tasks dynamically, and reviews outputs.	Complex tasks where a "boss" needs to coordinate

```
# Hierarchical process – needs a manager LLM
crew = Crew(
    agents=[researcher, writer, reviewer],
    tasks=[complex_task],
    process=Process.hierarchical,
    manager_llm="gpt-4o" # This LLM acts as the manager
)
```

YAML Configuration (The Professional Way)

For larger projects, CrewAI lets you define agents and tasks in YAML files — cleaner and easier to maintain:

```
# agents.yaml
stock_analyst:
  role: "Senior Stock Market Analyst"
  goal: "Identify undervalued stocks with high growth potential"
  backstory: "Expert analyst with 15 years on Wall Street..."

risk_assessor:
  role: "Risk Management Specialist"
  goal: "Evaluate investment risks and provide safety ratings"
  backstory: "Former hedge fund risk manager..."
```

```
# tasks.yaml
analyze_stocks:
  description: "Analyze the top 10 tech stocks. Look at P/E ratios, growth rates, and market position."
  expected_output: "A ranked list of 10 stocks with analysis for each."
  agent: stock_analyst

assess_risk:
  description: "For each stock identified, assess the investment risk level."
  expected_output: "Risk ratings (Low/Medium/High) for each stock with justification."
  agent: risk_assessor
```

Project 4: Stock Picker Agent

PROJECT 4

Stock Picker — AI Investment Research Team

What it does: A crew of agents that researches stocks, analyzes financials, assesses risk, and recommends investments.

Agents involved: Market Researcher, Financial Analyst, Risk Assessor, Report Writer

Key concepts: Sequential process, web search tools, financial data APIs

Project 5: 4-Agent Engineering Team

PROJECT 5

AI Engineering Team — Build Software with AI Agents

What it does: A full software development team made of AI agents that can design, code, test, and document software.

Agents involved: Product Manager, Software Engineer, QA Tester, Technical Writer

Special feature: Coder agents run in Docker containers for safe code execution

Key concepts: Hierarchical process, code execution tools, Docker integration

06 — Week 4: LangGraph

Build sophisticated, stateful agent workflows with full control

What is LangGraph?

The "sophisticated and powerful" framework — graph-based state machines for complex workflows

LangGraph

"Maximum control over your agent's behavior — like programming a flowchart"

LangGraph represents your agent workflow as a directed graph — nodes are actions, edges are transitions. You have complete control over every decision point, making it ideal for production systems that need reliability and auditability.

- ✓ Maximum control
- ✓ Best for production
- ✓ Built-in state persistence
- ⚠ More code to write

Understanding Graphs — The Core Concept

GPS NAVIGATION ANALOGY

Think of LangGraph like a GPS navigation system. The **nodes** are locations (actions to take). The **edges** are roads between them. The **state** is your current position and everything you know. **Conditional edges** are like "if traffic is bad, take the highway; otherwise take the local road." You define the entire map — the agent navigates it.

The 4 Core Concepts

Concept	What it is	Code equivalent
State	All the information the agent tracks (like memory)	A Python TypedDict or dataclass
Node	A step/action in the workflow (a function)	A Python function that takes state, returns updated state
Edge	A connection between nodes (always goes from A to B)	<code>graph.add_edge("node_a", "node_b")</code>

`graph.add_conditional_edges(...)`

Conditional Edge	A branching connection (goes to A or B based on logic)
-------------------------	---

Building a LangGraph Agent — Step by Step

```

from langgraph.graph import StateGraph, END
from typing import TypedDict, List

# ---- STEP 1: Define the State (what the agent remembers) ----
class AgentState(TypedDict):
    messages: List[str]          # Conversation history
    research_done: bool          # Has research been completed?
    final_answer: str            # The final output

# ---- STEP 2: Define Nodes (the actions) ----
def research_node(state: AgentState) -> AgentState:
    # Do research, update state
    research_results = "Found relevant information..."
    state["messages"].append(research_results)
    state["research_done"] = True
    return state

def answer_node(state: AgentState) -> AgentState:
    # Generate final answer based on research
    state["final_answer"] = "Based on research: ..."
    return state

# ---- STEP 3: Define routing logic ----
def should_research(state: AgentState) -> str:
    if state["research_done"]:
        return "answer" # Go to answer node
    else:
        return "research" # Go to research node

# ---- STEP 4: Build the graph ----
graph = StateGraph(AgentState)

# Add nodes
graph.add_node("research", research_node)
graph.add_node("answer", answer_node)

# Add edges
graph.set_entry_point("research") # Start here
graph.add_conditional_edges(
    "research",
    should_research,
    {"answer": "answer", "research": "research"}
)
graph.add_edge("answer", END) # End after answering

# ---- STEP 5: Compile and run ----

```

```
app = graph.compile()
result = app.invoke({"messages": [], "research_done": False, "final_answer":
""})
```

State Persistence — Agents That Remember

One of LangGraph's superpowers is **checkpointing** — saving the agent's state so it can resume if interrupted.

```
from langgraph.checkpoint.sqlite import SqliteSaver

# Save state to a database
checkpointer = SqliteSaver.from_conn_string("agent_memory.db")
app = graph.compile(checkpointer=checkpointer)

# Run with a thread ID (like a session ID)
config = {"configurable": {"thread_id": "user_123"}}
result = app.invoke(initial_state, config=config)

# Later, resume the same conversation!
result2 = app.invoke(new_message, config=config) # Remembers previous state
```

 **Why this matters:** Without checkpointing, if your agent crashes halfway through a 30-minute research task, you lose everything. With checkpointing, it picks up exactly where it left off.

Human-in-the-Loop

LangGraph makes it easy to pause an agent and ask a human for approval before continuing:

```
from langgraph.graph import interrupt

def approval_node(state):
    # Pause and ask human for approval
    human_decision = interrupt("Should I proceed with this action? (yes/no)")
    if human_decision == "yes":
        state["approved"] = True
    return state
```

Project 6: Sidekick Browser Agent

PROJECT 6

Sidekick — Your AI Browser Co-Pilot (like OpenAI Operator)

What it does: An agent that works alongside you in your browser — filling forms, clicking buttons, extracting data, and completing web tasks.

Key concepts: LangGraph state management, browser control tools, human-in-the-loop for sensitive actions

Real-world value: Automate repetitive web tasks — booking, form filling, data extraction

07 — Week 5: AutoGen (AG2)

Multi-agent conversations — agents that talk to each other



What is AutoGen?

Microsoft's framework for conversational multi-agent systems — agents collaborate through dialogue

AutoGen / AG2

"Agents that talk to each other — like a group chat for AI"

AutoGen's unique approach: agents coordinate by having conversations with each other. Instead of a fixed workflow, agents debate, ask questions, and collaborate dynamically through